

The Cortex-M port

How `micro-os-plus-iii-cortexm` is built, what its six boards are, and where its SMP lives.

This is the third architecture project, beside `-aarch32` and `-aarch64`. It is built the same way and from the same kernel, and it is the first port here with boards on **both** sides of the SMP line: three at `OS_NCPU=1` and three at `OS_NCPU=2` — and, on the RP2350 boards, with the line drawn per *test* rather than per board.

1. What it is

```
micro-os-plus-iii-cortexm/
├─ CMakeLists.txt
├─ include/cmsis-plus/rtos/port/      upstream's port contract
├─ include-rp2350/.../port/          os-c-decls.h, os-inlines.h: the SMP branch
├─ src/rtos/os-core.cpp              upstream: PendSV, SysTick, criticals
├─ src/rtos/os-core-rp2350.cpp       the same, plus the SMP branch
├─ src/semihosting-exit.cpp          strong _Exit() through SYS_EXIT
├─ test/
│   ├─ CMakeLists.txt               hw.sh  qemu.sh
│   ├─ boards/<id>/                 board.cmake, include/, src/, linker.ld,
│   │                               openocd.cfg, hw.sh  (pico2 also: glue/, usb/)
│   └─ <id>/                       that board's test applications
```

board	part	CPUs	tests
<code>nucleof411</code>	STM32F411RE, Cortex-M4F	1	4 — <code>mos-test1</code> + the 3 harness suites
<code>weactf411</code>	STM32F411CE, Cortex-M4F	1	5 — <code>mos-test1</code> , <code>spi-pipeline</code> + the 3 suites
<code>weactf412</code>	STM32F412RE, Cortex-M4F	1	5 — <code>mos-test1</code> , <code>uart-test1</code> + the 3 suites
<code>pico2</code>	RP2350, 2× Cortex-M33, 4 MB flash	2	15 — 12 port tests + the 3 suites
<code>pico2-rp2350b-psram</code>	RP2350B, 16 MB flash + 8 MB PSRAM	2	14
<code>pico2-pizero</code>	RP2350B, 16 MB flash, Pi-Zero form factor	2	14

Four are **hardware-only**: they set no `UOS_BOARD_LINKER_QEMU`, so each builds one image per test and `test/qemu.sh` answers by naming `hw.sh`. `pico2` and `pico2-rp2350b-psram` set it, and build a `-qemu` image for the tests QEMU's generic Cortex-M can run (`pico2`: `smp-test1`, `sc-test-ko` and the three suites; `pico2-rp2350b-psram`: `smp-test1`, `sc-test-ko`); every other test is listed `BOARD_TEST_HWD_ONLY`. What each test does is in `tests/TESTS-CATALOG.md`.

The three RP2350 boards

They are one piece of silicon and one set of board code. `pico2` owns all of it — `src/`, `glue/`, `include/`, `usb/` and the three linker scripts — and the other two `include` its `board.cmake` and then state only what differs. That works because `CMAKE_CURRENT_LIST_DIR` is per `file`: every path that file sets still resolves into *its* directory, not the includer's, so nothing is copied and nothing is repeated.

	<code>pico2</code>	<code>pico2-rp2350b-psram</code>	<code>pico2-pizero</code>
flash	4 MB	16 MB	16 MB
UART	GPIO0/1	GPIO12/13	GPIO0/1
LED	GPIO25	GPIO25	GPIO5
arch define	<code>__ARM_ARCH_7EM__</code>	<code>__ARM_ARCH_7EM__</code>	<code>__ARM_ARCH_8M_MAIN__</code>
PSRAM	—	window 1 @ <code>0x11000000</code> , CS GPIO0	—
probe	any CMSIS-DAP (no vid_pid pinned) @1000	<code>0xc251:0xf001</code> @1000	<code>0x0416:0x5951</code> @2000

The arch define is the one that is not wiring. The M33 **is** ARMv8-M Mainline, and the predecessor's Pi-Zero Makefiles said so, while every other board on this silicon forces the v7E-M path. They select different branches of the port headers, and the two have never been compared on the desk. Read a result from `pico2-pizero` with that in mind.

Their test sets differ because the predecessor's did:

- all three carry `smp-test1` ... `smp-test5`, `smp-mat-test`, `sc-test-ko`, `smp-test-ko`, and the two kernel-less probes `smp-test0` and `exc-test`
- `pico2` adds the two **USB gadget** tests, because both link the 4 MB script
- `pico2-rp2350b-psram` adds `smp-test-nested` and the three `smp-test-nested-clock` variants, which exist in the predecessor for the 16 MB part alone

Not carried, deliberately: everything under `*-loader-*` and `*xip*` (they need fixture images), the `mini-a` board (a USB-CDC console and a WS2812 in place of the UART and the LED), and `smp-test-mini-a-usb-cdc-acm_agy` — `main.cpp` and `Makefile` are byte-identical to the test that *is* carried.

2. Where the SMP lives — the board, not the port

The design spec framed step 4 as *merging* pico2's dual-core core with the STM32 boards. Reading pico2's core changed that, and the new shape is better:

pico2's SMP is RP2350 silicon.

kernel lock	SIO hardware spinlock 0 . Not LDREX/STREX: the RP2350 has no global exclusive monitor, so the architectural route does not work across the two cores.
IPI	the SIO inter-core FIFO on <code>SIO_IRQ_FIFO</code> (external IRQ 25). The handler drains the FIFO and pends its own core's PendSV.
CPU index	<code>SIO_CPUID</code> .

None of that is Cortex-M — it is one SoC's. In this layout that makes it board code, exactly as the Lyra's GIC-400 SGI and the Pi's are board code. So `cortexm` did not gain SMP by changing its ISA file; it gained SMP because a board arrived that supplies a lock and an IPI.

Core 1 is launched through the bootrom FIFO handshake with its own MSP. Only core 0 advances the RTOS clock; core 1's SysTick reschedules core 1 alone.

Proof it is really in the image

Symbols from the linked `smp-test2-hwd`, not from the build log:

```
T port_cpu_id                T SIO_IRQ_FIFO_Handler
T PendSV_Handler             T SysTick_Handler
D os::rtos::port::scheduler::_smp_klock
T multicore::launch_core1(void (*)(), unsigned long)
b os::rtos::port::scheduler::core1_msp_stack

B os::rtos::port::scheduler::lock_state      size 2
B os::rtos::scheduler::os_idle_thread_core  size 8
```

`lock_state` is `state_t[OS_NCPU]` and measures **2** bytes; `os_idle_thread_core` is `thread*[OS_NCPU]` and measures **8**. `OS_NCPU=2` reached the scheduler.

3. Two port cores, and why they are not one

`src/rtos/os-core.cpp` is upstream's, single-core. `src/rtos/os-core-rp2350.cpp` is the predecessor's pico2 core: **the same file plus an** `OS_USE_SMP_SCHEDULER` **branch**, 91% identical by line. Its port

headers already handle every M profile — `__ARM_ARCH_6M__`, `7M`, `7EM`, `8M_MAIN` — and every RP2350 reference in them is inside the SMP branch.

So they *could* be one file. They are not, for a reason rather than an oversight: **the three STM32 boards are hardware-proven on upstream's core**. Moving them onto a different 748-line scheduler is a change that has to be re-proven on the desk, not assumed — and the standing rule here is that adding a board must not reach boards already tested.

A board therefore names the core it was proven with:

```
set (UOS_BOARD_PORT_CORE      "src/rtos/os-core-rp2350.cpp")
set (UOS_BOARD_PORT_INCLUDE  "include-rp2350;include")
```

`include-rp2350/` holds only the two headers that differ (`os-c-decls.h`, `os-inlines.h`, with the SIO spinlock and `SIO_CPUID`). It is searched first, and `os-decls.h`, the same code for both cores, comes from `include/`.

Defaults are upstream's, so a board that says nothing gets what the STM32 boards get. Merging the two is a later, explicit step.

`pico2` forces `-D__ARM_ARCH_7EM__` although the M33 is ARMv8-M, as the predecessor's Makefiles did. The v7E-M path is the one that board was brought up on; letting the compiler select `__ARM_ARCH_8M_MAIN__` would change the scheduler under a working board.

4. What else became a board choice

Two things the port used to hard-code, because the boards disagree:

	STM32F4	pico2
startup	<code>micro-os-plus::iii-startup</code> — upstream's generic startup	its own <code>boot.S</code> + <code>interrupt-vectors.S</code>
trace backend	<code>micro-os-plus::iii-trace-semihosting</code>	its own <code>trace-uart.cpp</code> , over the UART

Both are named in each board's `UOS_BOARD_LIBS`. This is why the kernel splits them out as separate targets in the first place.

5. `src/` and `glue/` on the RP2350 boards

pico2 is the only board whose sources split in two, and the split is read off the predecessor's Makefiles, not invented:

src/	boot.S, interrupt-vectors.S, init-fini-stubs.c, clocks.cpp, led.cpp — every Makefile listed them
glue/	uart.cpp, rtos-glue.cpp, trace-uart.cpp, syscalls.c, multicore.cpp, psram.cpp, adc.cpp, ws2812.cpp — the Makefiles disagree, one test at a time
usb/	TinyUSB and its two class drivers, for the two gadget tests

Who takes what, and why:

- smp-test5 defines its own uart:: in main.cpp, so it must not also get the board's uart.cpp.
- smp-test1 and sc-test-ko are **single-core**: no multicore.cpp.
- the four PSRAM tests take psram.cpp; the three nested-clock tests take adc.cpp; the CDC test takes ws2812.cpp.
- smp-test0 has no RTOS at all, so no glue, no trace, no syscalls.

A glob over src/ linked both copies and failed on duplicate definitions — uart::init(), uart::put_char(), HardFault_Handler. Each board's test/<board>/tests.cmake composes them per test through the board_test_sources() hook, which is the same hook the Lyra uses for the same reason.

TinyUSB, carried once

The predecessor kept two copies of TinyUSB, src/usb-cdc/ and src/usb-hid/, byte-identical apart from the one class driver each keeps. Here it is one tree under boards/pico2/usb/ carrying both classes, and only what genuinely differs stays split — tusb_config.h, usb_descriptors.c and the class glue, in usb/cdc/ and usb/hid/.

tusb_config.h is why the per-test include directory has to come **first**: TinyUSB includes it by plain name, and the two tests configure different device classes. The images show it worked — the CDC image carries eight cdc_* symbols and no hidd_*, the HID image the reverse.

Three facts that are per test, not per board

The RP2350 boards needed three hooks that no earlier board did. All three are in the shared test loop, default to "the board's own answer", and are therefore invisible to every board that does not use them:

hook	what it decides	who uses it
board_test_ncpu()	how many CPUs one test runs on	smp-test1, sc-test-ko at OS_NCPU=1 on a two-core board
board_test_linker()	a linker script instead of the board's	the four PSRAM tests
board_test_options()	compile options on the test's own sources	the four nested-interrupt tests
BOARD_TEST_NO_KERNEL	that a test compiles no kernel at all	smp-test0, exc-test

The first replaced a mistake: `OS_USE_SMP_SCHEDULER` used to be a board-wide define, and the predecessor's `smp-test1` (the single-core RTOS bring-up) and `sc-test-ko` (the single-core kernel-object test) define it in no Makefile. `uos_add_app()` derives the define from `NCPU`, so the hook is the whole fix. `sc-test-ko` matters more than its name suggests: it is the only test on this silicon that exercises the kernel's **non-SMP** branch.

The second is visible in the ELF. On `pico2-rp2350b-psram`, `smp-mat-test` has 1.27 MB of `.bss` and a 128 KB heap at `0x11000000` — external PSRAM — while `smp-test2`, on the same board, has everything at `0x20000000`.

The third is a build failure waiting for anyone who skips it. The nested tests place their interrupt handlers in `.data` so the code runs from RAM instead of XIP flash, and their Makefiles compiled `main.cpp` — only `main.cpp` — with `-g0 -mlong-calls`. Both halves are load bearing: a call from flash to a `.data` function is out of a Thumb `BL`'s reach, and GCC's location views cannot describe a function whose section moved. Without it the assembler stops with `Error: leb128 operand is an undefined symbol: .LVU59`.

Two tests that compile no kernel

`smp-test0` and `exc-test` are the bare-metal probes that run *before* a scheduler exists — `smp-test0` starts the second core itself and checks the two can talk, `exc-test` asks whether exception **entry** works at all. Their Makefiles list nine files and seven files, and not one is the kernel. Their value is precisely that they run without one, so they are not to be rewritten as RTOS tests.

Expressing that took a split in the port. `micro-os-plus::cortexm` is now the **board** half — flags, include path, start-up, silicon support, all of it true before any scheduler — plus the kernel and the ISA scheduler on top. The board half is exported on its own as `micro-os-plus::cortexm-bare`, a test named in `BOARD_TEST_NO_KERNEL` links that instead, and the shared `uos_add_app()` grew a `NO_KERNEL` option that skips the kernel and `OS_NCPU` — there is nothing there to configure.

It is visible in the images: `smp-test0` and `exc-test` carry **zero** `os::rtos` symbols and no `_Exit`, at 2.6 KB and 2.3 KB of text, where `smp-test2` on the same board carries 145 and 21.7 KB.

A port that has no such test simply leaves `UOS_PORT_BARE_LIB` unset, and the test loop refuses the request rather than quietly linking a kernel.

6. Running them

OpenOCD only. No GDB anywhere in this path, and nothing redirected.

```
BOARD=nucleof411      ./test/hw.sh          # list what is built
BOARD=nucleof411      ./test/hw.sh mos-test1
BOARD=pico2           ./test/hw.sh smp-test2
BOARD=pico2-rp2350b-psram ./test/hw.sh smp-mat-test
BOARD=pico2-pizero    ./test/hw.sh sc-test-ko
```

The STM32 boards: program, `arm semihosting enable`, `reset run`. Semihosting must be enabled **before** the image runs — `os::trace::printf()` issues `BKPT 0xAB`, which faults if the debugger is not listening for it.

The three RP2350 boards are different in two ways, both learned on the board and both carried in each `hw.sh`:

- `reset init`, not `reset`. A bare reset does not re-run the RP2350 bootrom/XIP setup, so the freshly programmed image never boots.
- **Resume `cm1` before `cm0`**. With `USE_SMP 0` the rp2350 target exposes two targets and `reset init` halts both, while a single `resume` resumes only the current one. A debug-halted core 1 will **not** boot from the software PSM reset `launch_core1()` issues — the launch then hangs forever waiting for the bootrom readiness word. Resuming `cm1` first leaves it in the bootrom core-1 launch loop, where it answers the FIFO handshake.

They trace over their own UART, not semihosting, so OpenOCD exits once both cores are running and the console is the board's serial port — on whichever pins that board wires it to.

7. Strong `_Exit()`

The same behaviour as the two A-profile ports, with this ISA's trap. The kernel's `_Exit()` is weak, so `src/semihosting-exit.cpp` overrides it and ends the run through semihosting `SYS_EXIT` rather than idling.

Nothing about the trap is written in this port. The kernel already ships `cmsis-plus/arm/semihosting.h`, which selects `bkpt` from `__ARM_ARCH_7M__` / `7EM` / `6M`, so this file calls its `report_exception()`. There is no `semihosting.hpp` here: the AArch32 one exists because on A-profile the trap depends on which OpenOCD *target type* drives the core (`cortex_a` → `SVC`, `aarch64` → `HLT`), a choice that does not arise on Cortex-M.

Verified in the linked image:

```
080038f4 T _Exit          <- strong, overriding the kernel's weak one
      movs r4, #24        <- 0x18 = SYS_EXIT
      bkpt 0x00ab
      .word 0x00020026     <- ADP_Stopped_ApplicationExit
      .word 0x00020023     <- ADP_Stopped_RunTimeError
```

8. What is not done

- **Hardware results are not recorded here.** When this section was first written nothing had run on a board. Since then every board has a `-hwd` runner (`test/boards/<id>/hw.sh`, with the `hw_result`

verdict), and the git history of `micro-os-plus-iii-cortexm` shows tests adjusted on hardware (e.g. the HID test's Escape-to-PASS). This document does not keep a per-test hardware record.

- **`sc-test-ko` is the pair's missing half, and it *is* carried.** Together with `smp-test-ko` it is the same ten kernel objects run single-core and cross-core, which makes it the closest thing this port has to a regression suite once a board is on the desk.
- **The other three pico2 trees.** `pico2-std` (Peterson's algorithm in software), `pico2-sdk` (the Pico SDK) and `pico2-sdk-min` (**byte-identical** port files to `pico2-sdk` — not a separate port). They differ from `pico2` in the kernel lock and nothing else, so each is a sibling board when wanted.
- **Merging the two port cores** (§3).
- **The rest of pico2's 36 application directories** — the XIP loaders and the `*xip*` set, which need fixture images, plus the `mini-a` board's own pair. None of them is about SMP.